

Now, watch the list summary in your app. See the total number of items being updated. Nowadays, we do not check the “done” data in any situation. However, if you are trying to check or uncheck the items, you will identify a bug.

• Now tracking the changes and approaching to be “done.”

You can capture the checkbox and update or manage the checklist accordingly. To trigger the change, attach **@change** to each check box instead of using a **v-model**.

Now, update the element and add:

- `<input>` in `ToDoItem.vue`

Since the checkbox was checked and we need to emit it, you can include in the line:

- `$emit()`
- `<input type="checkbox" class="checkbox" :id="id" :checked="isDone"`
- `@change="$emit('checkbox-changed')"` `>`

Now, add a new method called **updateDoneStatus()** in the `App.vue` below **addToDo()**. This would take one parameter, i.e., **todo** item id. Now, we want to find the item and match the id and update it to be “done” as of its current status.

Now, add a new method below your method. This method should take one parameter: the **todo** item id. We want to find the item with the matching id and update its done status as the opposite of its current status.

Now, we want to run the method with **ToDoItem** to emit a checkbox- to change the event and pass in its **item.id** as the parameter. Now, update your **<to-do-item>** that will be seen as:

- `updateDoneStatus(toDoId) {`
- `const toDoToUpdate = this.ToDoItems.find(item => item.id === toDoId)`
- `toDoToUpdate.done = !toDoToUpdate.done`
- `}`
- `<to-do-item :label="item.label" :done="item.done" :id="item.id"`
- `@checkbox-changed="updateDoneStatus(item.id)">`
- `</to-do-item>`